

Technical Documentation
JAVA VIDEO CREATOR

1. Project Overview

This is a console-based Java application that takes a set of photos and short videos as input and automatically produces a narrated in portrait mode video.

The program does the following, in order:

- Reads GPS coordinates and capture dates from each file's EXIF metadata.
- Sorts the files from oldest to newest.
- Sends the images to GPT-4o Vision to analyze what's literally visible.
- Generates a image with DALL-E 3 based on that visual analysis.
- Generates an inspirational phrase using the GPS coordinates of the first and last file.
- Produces a narration script (also via GPT-4o Vision) describing each image in order, closing with the map phrase.
- Converts the script to audio using OpenAI TTS-1.
- Builds a map slide from OpenStreetMap tiles with two pins (start and end).
- Put everything together into a final portrait video using FFmpeg.

External dependencies: OpenAI API (GPT-4o, DALL-E 3, TTS-1), FFmpeg, ExifTool, and the OpenStreetMap tile server.

2. Architecture

The project is split into 7 classes and 1 interface. Each class handles exactly one responsibility; nothing overlaps.

Class / Interface	Type	Responsibility
Main	class	Entry point. Handles all console I/O and calls each pipeline step in order.
MediaFile	class	Represents a single media file. Holds path, GPS, and capture date extracted on construction.
Exiftool	class	Wraps exiftool CLI calls to extract GPS and CreateDate from any media file.
SortMedia	class (static)	Sorts MediaFile arrays by date, identifies oldest/newest, detects video extensions.

OpenAICall	class	All communication with OpenAI: Vision, DALL-E 3, GPT-4o text, TTS-1.
MapBuilder	class	Downloads OSM tiles, stitches them, draws pins, overlays the phrase, saves the map PNG.
VideoCreator	class	All FFmpeg logic: create parts, concatenate, attach audio, convert to portrait.
MediaOperation	interface	Single method contract (execute) that VideoCreator implements.

Design Decisions

Interface as contract. MediaOperation declares a single execute(MediaFile[]) method. This keeps Main clean — it just calls videoCreator.execute(media) without knowing the internals.

Metadata extraction on construction. MediaFile calls Exiftool in its constructor, so by the time the object exists, GPS and date are already populated. No lazy loading, no null checks scattered around.

No JSON library. All API responses are parsed manually with a char-by-char walker. This avoids adding external dependencies to the project. It's a bit verbose but it works fine for the fixed response shapes that OpenAI returns.

Map built from tiles, not a static API. MapBuilder fetches individual 256x256 PNG tiles from tile.openstreetmap.org and stitches them into a canvas using Graphics2D. This avoids rate limits and API key requirements from services like Google Static Maps.

3. Class Reference

MediaFile

Represents one media file. All fields are populated in the constructor

Field	Type	Description
mediaPath	String (final)	Absolute or relative path to the file on disk.
GPS	String	GPS coordinates as returned by exiftool, "No data" if missing.
date	String	Capture date in "yyyy:MM:dd HH:mm:ss" format. "No data" if missing.

Exiftool

Thin wrapper around the exiftool CLI. Uses the `-s3` flag to get only the raw value with no label or formatting.

Method	Returns	Notes
<code>extractGPS(path)</code>	String	Runs <code>exiftool -s3 -GPSPosition</code> on the file.
<code>extractDate(path)</code>	String	Runs <code>exiftool -s3 -CreateDate</code> on the file.
<code>runExiftool(path, tag)</code>	String (private)	Starts the process, reads one line of stdout, returns "No data" if empty.

SortMedia

All static utility methods

Method	Description
<code>sortByDate(MediaFile[])</code>	Sorts the array in-place from oldest to newest. Files with missing dates fall to the front (<code>LocalDateTime.MIN</code>).
<code>getOldest(MediaFile[])</code>	Returns <code>media[0]</code> — assumes the array has already been sorted.
<code>getNewest(MediaFile[])</code>	Returns <code>media[media.length - 1]</code> .
<code>forVideo(String path)</code>	Returns true if the path ends in <code>.mp4</code> , <code>.mov</code> , <code>.avi</code> , <code>.mkv</code> , <code>.webm</code> , or <code>.m4v</code> .

OpenAICall

Handles all communication with the OpenAI API.

Response Parsing

OpenAI wraps text replies inside `choices[0].message.content` as a JSON string. The `extractTextFromResponse` method walks the string char by char to find the real closing quote, correctly handling escaped sequences like `\`, `\n`, and `\\` that are part of the content value. A naive `indexOf` approach breaks when the content ends with a letter before a brace.

DALL-E 3 responses are parsed by `extractBase64Image`, which looks for the `"b64_json"` marker and walks forward to the next unescaped quote. It tries both `"b64_json": "` (with space) and `"b64_json":` (without) to handle format changes between OpenAI API versions.

MapBuilder

Builds the closing map image entirely from OSM tiles using `Graphics2D`.

Tile Assembly

The buildMap method converts the midpoint between the two GPS coordinates to fractional tile coordinates, then downloads all tiles needed to cover the 1080x1440 canvas. Each tile is a 256x256 PNG fetched from tile.openstreetmap.org. Tiles are drawn onto a BufferedImage with the correct pixel offsets so the center coordinate lands exactly in the middle.

GPS Parsing

ExifTool returns GPS strings in DMS format. The parseDMSToDecimal method strips all non-numeric characters and tokenizes by whitespace to extract degrees, minutes, seconds, and direction. South and West coordinates are returned as negative decimal degrees.

Zoom Selection

computeZoom looks at the maximum difference in latitude or longitude between the two points and returns a zoom level between 3 (continental) and 14 (street level). This keeps both pins visible in the frame without any manual configuration.

Phrase Overlay

overlayPhrase draws a semi-transparent black banner (170/255 alpha) over the bottom portion of the map. The phrase is rendered in white italic SansSerif, centered horizontally and vertically within the banner. Long phrases are automatically word-wrapped using wrapText, which splits on word boundaries so no word gets cut.

Fallback

If neither file has GPS data, or if the tile fetch fails completely, createFallbackMapImage generates a plain dark image with a "No location data available" label. The phrase is still overlaid at the bottom so the closing slide always exists for the video assembly.

VideoCreator

Handles all FFmpeg operations. Implements MediaOperation, so Main just calls execute(media).

Image Parts

createImagePart uses -loop 1 to hold a still image for the specified duration. A blurred version of the image is scaled up to fill 1920x1080 as the background, and the original is scaled down to fit within that frame and overlaid centered. This avoids black bars on any aspect ratio.

Video Parts

createVideoPart uses the same blur-background approach but reads from an actual video file. The -an flag strips the original audio since the narration track replaces everything.

Audio Normalization

compileFinalVideo applies loudnorm=l=-14:TP=-1:LRA=7 to normalize the narration to -14 LUFS / -1 dBTP / 7 LRA. These are standard values for online video platforms. The verifyAudioLevels method re-runs FFmpeg in analysis mode and prints the actual measured values so you can confirm normalization worked.

Portrait Conversion

convertToPortrait applies the same blur-background technique but to a 1080x1920 canvas. The horizontal final_video.mp4 becomes the source. Audio is copied without re-encoding using -c:a copy.

4. Pipeline Flow

The following is the full execution order as orchestrated by Main.java:

Step	Method called	Output artifact
1	api.analyzeVisualContent(media, key)	visualDescription (String)
2	api.generateImage(essencePrompt, key, "Ailimage.png")	Ailimage.png
3	api.generateText(api.MapPhrasePrompt(oldest, newest), key)	phrase (String)
4	api.generateNarrationScript(media, key, phrase)	DescriptionImages (String)
5	api.TextToSpeech(DescriptionImages, key, "audio.mp3")	audio.mp3
6	new MapBuilder().buildMap(oldest, newest, phrase, "mapImage.png")	mapImage.png
7	videoCreator.execute(media)	final_video_portrait.mp4

5. File Structure

File	Description
Main.java	Entry point. Console I/O and pipeline orchestration.
MediaFile.java	Data class for a single media file with its EXIF metadata.
Exiftool.java	CLI wrapper for extracting GPS and date from files.
SortMedia.java	Static utilities for sorting and querying the media array.
OpenAICall.java	All OpenAI API calls: Vision, DALL-E 3, GPT-4o text, TTS-1.
MapBuilder.java	OSM tile assembly, pin drawing, phrase overlay.

VideoCreator.java	All FFmpeg operations for video assembly and portrait conversion.
MediaOperation.java	Interface with a single execute(MediaFile[]) method.
AilImage.png	Generated at runtime. It is a AI cover image from DALL-E 3.
audio.mp3	Generated at runtime. Its a TTS narration audio.
mapImage.png	Generated at runtime. The closing map image.
final_video.mp4	Intermediate. The horizontal video before portrait conversion.
final_video_portrait.mp4	Final output. portrait video with narration.
input.txt	Temporary FFmpeg concat list. Deleted after assembly.

6. Error Handling

Situation	Handling
OpenAI API call fails	analyzeVisualContent falls back to a generic description. generateNarrationScript falls back to metadata-based narration. Other calls print the HTTP status and continue.
All media files are videos (no images)	generateNarrationScript calls generateNarrationFromMetadata, which builds a prompt from date + GPS strings instead of sending images.
GPS missing from a file	MapBuilder checks oldest.GPS and newest.GPS. If one is missing, it reuses the other for both pins. If both are missing, createFallbackMapImage is called.
OSM tile fetch fails	That tile is skipped (grey fallback color is used). If every single tile fails, assembleTiles returns null and buildMap falls back to the plain dark image.
FFmpeg fails	runCommand throws a RuntimeException with the exit code. Main catches it, prints the message, and exits early.
MediaFile path is null or empty	MediaFile constructor throws RuntimeException immediately.
DALL-E or TTS call fails	The PNG or MP3 is not saved. VideoCreator logs a warning if the file is missing when creating the video part and continues rather than crashing.
OpenAI token not set	Main checks System.getenv("OpenAIToken") on startup and exits with a message if it's null.